ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

COMPUTER ARCHITECTURE

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 2nd year Computer Architecture paper 1 · practice a.

Written by Academic Central for practice, following second-year Computer Science syllabuses at South African universities (computer organisation and architecture). It is not a university memorandum.

How to mark: each ✓ is one mark. Give marks for correct working even if the final answer is wrong (consistent accuracy). Accept other correct methods and notation (e.g. $\overline{A}$ or ¬A for A′, + or ∨ for OR).

QUESTION 1

1.1 B ✓

1.2 C ✓

1.3 B ✓

1.4 C ✓

1.5 B ✓

1.6 A ✓

1.7 C ✓

1.8 C ✓

1.9 A ✓

1.10 B ✓

(10)

[10]

QUESTION 2

2.1 156 = 128 + 16 + 8 + 4 (or repeated division by 2) ✓ = 10011100_2 ✓ (4)
1001 1100 = 9 and C ✓ = $9C_{16}$ ✓

2.2 8 + 4 + 1 + 0.5 + 0.125 ✓ = 13.625 ✓ (2)

2.3 45 = 00101101 ✓ invert: 11010010 ✓ add 1: 11010011 ✓ (3)

2.4 01100100 + 01010000 = 10110100 ✓ (3)
Overflow: yes ✓. Two positive numbers (100 + 80 = 180) gave a negative result; 180 is more than the largest 8-bit value, 127 ✓

6.75 = 110.11_2 ✓ = 1.1011×2^2 ✓
Sign = 0 ✓
Exponent = 127 + 2 = 129 = 10000001 ✓
Mantissa = 10110000000000000000000 ✓

(5)

2.6 0100 0000 1101 1000 0000 0000 0000 0000 ✓ = $40D80000_{16}$ ✓ (2)

2.7 Hex is much shorter and easier to read, and each hex digit stands for exactly 4 bits ✓ (1)

[20]

QUESTION 3

3.1 (4)

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

All 8 input rows ✓; rows with A = 0 correct ✓✓; rows with A = 1 correct ✓

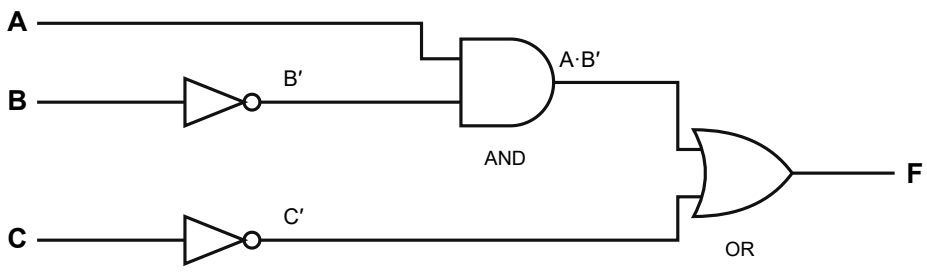
3.2 (4)

A·B + A·B′ = A·(B + B′) distributive law ✓
= A·1 = A complement and identity laws ✓
A + A′·B = (A + A′)·(A + B) distributive law ✓
= A + B ✓

3.3 (5)

K-map with 1s in cells 000, 010, 100, 101, 110 ✓
Group of four: m0, m2, m4, m6 = C′ ✓✓
Group of two: m4, m5 = A·B′ ✓
F = C′ + A·B′ ✓

3.4



NOT gates on B and C ✓; AND gate for A·B′ ✓; OR gate combining A·B′ and C′ ✓

(3)

3.5 (4)

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum column ✓ Carry column ✓
Sum = A ⊕ B (= A′B + AB′) ✓ Carry = A·B ✓

[20]

QUESTION 4

4.1 Any FOUR: Program counter (PC): address of the next instruction ✓; Memory address register (MAR): address of the memory location being read or written ✓; Memory data register (MDR): data going to or coming from memory ✓; Instruction register (IR): the instruction being decoded and executed ✓; Accumulator (ACC): results of the arithmetic and logic unit; Status register: flags such as zero, carry and overflow (4)

4.2 MAR ← PC ✓
MDR ← M[MAR] (read memory) ✓
PC ← PC + 1 ✓
IR ← MDR ✓
The control unit decodes the instruction in IR and executes it (e.g. fetches the operand, the ALU does the operation, the result is stored) ✓ (5)

4.3 ACC: 7, 12, 12, 8, 20, 20 ✓✓
M[22] = 12 ✓ M[24] = 20 ✓ (4)

4.4 Any FOUR: RISC has fewer, simpler instructions, CISC has many complex ones ✓; RISC instructions have a fixed length, CISC variable lengths ✓; RISC instructions mostly take one clock cycle, CISC instructions take several ✓; RISC is load/store (only LOAD and STORE use memory), CISC instructions can work on memory directly ✓; RISC has more registers; RISC is easier to pipeline; RISC needs more instructions per program (4)

4.5 Pipelining overlaps the stages of several instructions, so that while one is executing the next is being decoded and another fetched ✓. Hazard, any ONE with explanation: data hazard, an instruction needs the result of one that has not finished ✓✓; control hazard, a branch means the wrong instructions were fetched; structural hazard, two stages need the same hardware at once (3)

[20]

QUESTION 5

5.1 Registers, cache, main memory, hard disk or SSD ✓✓
Faster memory is much more expensive per byte, so a computer has a little very fast memory and a lot of cheap, slow memory ✓; keeping the data in use in the fast levels gives nearly the speed of the fastest memory at the cost of the cheapest ✓ (4)

5.2 Temporal locality: data used recently is likely to be used again soon ✓, e.g. a loop counter or the instructions inside a loop ✓
Spatial locality: data near recently used data is likely to be used soon ✓, e.g. going through an array element by element ✓ (4)

5.3 Offset: 16 bytes = 2^4, so 4 bits ✓✓
Index: 64 lines = 2^6, so 6 bits ✓✓
Tag: 32 − 4 − 6 = 22 bits ✓✓ (6)

5.4 Hit ratio = 950 / 1 000 = 0.95 ✓
Effective access time = 0.95 × 2 + 0.05 × (2 + 50) ✓ = 1.9 + 2.6 ✓ = 4.5 ns ✓ (4)

5.5 Virtual memory uses part of the disk as if it were extra main memory, with addresses translated in pages ✓. It lets programs larger than the RAM run, and gives each process its own protected address space ✓ (2)

[20]

QUESTION 6

6.1 Address bus: carries the memory or I/O address, from the CPU ✓
Data bus: carries the data in both directions ✓
Control bus: carries control signals such as read, write, clock and interrupt ✓ (3)

6.2 Polling: the CPU keeps checking each device to see whether it needs attention, which wastes CPU time ✓✓
Interrupts: the device sends a signal to the CPU only when it needs attention, so the CPU can do other work meanwhile ✓ (3)

6.3 A DMA controller moves blocks of data directly between a device and memory without the CPU copying each byte ✓; the CPU is free to do other work and is told by an interrupt when the transfer is finished ✓ (2)

6.4 2^{32} ✓ = 4 294 967 296 bytes = 4 GiB ✓ (2)

[10]

TOTAL: 100

END OF MARKING GUIDELINE